NexusIQ — Plataforma de Mantenimiento Predictivo Inmutable 🧠

Una API basada en **PostgreSQL, FastAPI y Python**, un motor de lógica distribuida para *Event Sourcing*, ingesta asíncrona tolerante a fallos locales, digital twins estáticos en tiempo real y *Correlation Insights* para Data Science, con 19 pruebas atómicas end-to-end con paralelismo y testing temporal aislado.

🚀 Requerimientos Técnicos

- **Docker y Docker-Compose**
- Opcionalmente `Python 3.11+` si decides no correrlo contenedorizado.

📦 Inicialización Ultra-Rápida con Docker Compose

El proyecto está diseñado para ser completamente automatizado desde que empieza su ejecución local. Todo el *stack*, incluyendo la persistencia del clúster de base de datos JSONB y la ingesta masiva inicial de datos, ocurren dentro de:

```
# ¡Hazlo todo de una vez! Contenedores aislados montan y empiezan la UI con los datos
docker-compose up -d --build
```

> 👉 *Al inicializarse por primera vez, el pipeline procesará unas 10.000 filas provistas en la carpeta `data/ai4i2020.csv`.* Las subsecuentes veces que interrumpas el proceso usarán las UUID criptográficas para ignorar idénticas escrituras de forma natural.

🧪 Pruebas Automatizadas Unitarias En Aislamiento (`pytest`)

Toda la lógica crítica (`detrending, idempotencia UDI, time-travel temporal` y la UI del backend) están rigurosamente probadas inyectando instancias 100% aleatorias temporalizadas sin requerir depurar por mano propia tu Base de Datos:

```
# Correr tests con reporte corto y traceback.
docker-compose run --rm app pytest tests/ -v --tb=short
```

🔌 API y Visualización de Componentes (`/health/view`)

La mejor manera de sentir en profundidad la plataforma funcionando es consumiendo sus Endpoints.

1. **La Interfaz Interactiva Web "NexusIQ Telemetry"**: Hemos proveído de forma directa en Fastapi un frontend CSS puro minimalista y Glassmorphic:
   Visita localmente **http://localhost:8000/health/view**, observando métricas como los **Insights descubiertos**, gemelos desactualizados y milisegundos de *Lag*.

2. **Endpoints de la Arquitectura Backend**:
   Adicionalmente a navegar por Swagger OpenAPI **(http://localhost:8000/docs)**, los componentes REST vitales incluyen:
   - `GET /health` (`status` del Event-sourcing y número de datos en RAW JSON)
   - `GET /health/view` (UI Minimalista interactiva / Dashboard HTML)
   - `GET /events` (Devuelve todos los eventos, con paginación `limit` y `offset`, filtrables por `domain`, `entity_id` y timestamps)
   - `GET /events/{event_id}` (Devuelve un evento puntual buscado por su UUID)
   - `GET /twins/{machine_id}` (Digital Twin puro con formula DS o default)
   - `GET /twins/{machine_id}/at?timestamp=YYYY-MM-DD...` (Time-Travel estático temporal)
   - `GET /correlations` (Narrativas y métodos predictivos en Español filtrados por p_value de Bonferroni ajustados)
   - `POST /pipeline/run` (Dispara la ingesta de forma asíncrona manual, devuelve Job ID)

🗂 Directorio / Estructura Analítica (Backend Engineering & Data Science)

Se organizó la prueba modularmente:

- `/src/pipeline/` (`M1`): Motor de inserción masivo. Hasheos `UUID5`.
- `/src/twins/` (`M2`): Motor gemelo digital (M2). Mantiene promedios a perpetuidad evitando latencia en frontend (+ Time Travel).
- `/src/correlations/engine.py` (`M3`): Script para tu compañero DS donde convive código de `SciPy` y `Scikit-Learn` sobre Datasets originarios de consultas SQl.
- `/tests/` : Compleja inyección `SessionLocal` y UUID aleatorios aislantes.
- `/src/templates/`: Front end dinámico.